



SAPIENZA
UNIVERSITÀ DI ROMA

Distributed Systems

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente

Prof. Alessandro Mei

Appunti di

Alessio Cavedal

GitHub: <https://github.com/AlessioCavedal>

Anno Accademico 2026/2027

Indice

1	Sistemi distribuiti e computazioni	1
1.1	Il problema: valutare un predicato globale	1
1.2	Il modello: sistemi distribuiti asincroni	2
1.3	Computazioni distribuite	3
2	Stati globali, tagli e consistenza	6
2.1	Stati globali, tagli e run	6
2.2	Monitoraggio attivo e deadlock fantasma	7
2.3	Tagli e run consistenti	8
2.4	Il reticolo degli stati globali	9
3	Osservare una computazione: orologio reale e clock logici	12
3.1	Monitoraggio passivo e osservazioni	12
3.2	Regole di consegna e consegna FIFO	13
3.3	Orologio reale e clock condition	13
3.4	Clock logici di Lamport	14
3.5	Consegnare le notifiche in ordine di clock logico	16
4	Consegna causale e vector clock	17
4.1	Consegna causale	17
4.2	La strong clock condition	18
4.3	Storie causali	18
4.4	Vector clock	19
4.5	Confrontare i vector clock	21
4.6	Consegna causale con i vector clock	24
5	Snapshot distribuiti	26
5.1	Stati dei canali	26
5.2	Primo protocollo: un orologio reale	26
5.3	Dall'orologio reale al protocollo di Chandy-Lamport	27
5.4	Il protocollo di Chandy-Lamport	28
5.5	Perché funziona, e perché servono i canali FIFO	29
5.6	Proprietà dello snapshot	30
5.7	A cosa serve uno snapshot	31

Capitolo 1

Sistemi distribuiti e computazioni

Molti problemi dei sistemi distribuiti hanno la stessa forma: bisogna accorgersi che lo stato complessivo del sistema soddisfa una certa condizione e, quando succede, reagire. Per affrontarli serve prima un modello preciso di che cosa sia un sistema distribuito e di come si svolga una computazione al suo interno. Questo capitolo fissa il modello usato nei capitoli seguenti: processi che comunicano solo con messaggi, senza un orologio comune, e un ordine tra gli eventi basato soltanto su cause ed effetti.

1.1 Il problema: valutare un predicato globale

Ecco alcuni problemi tipici dei sistemi distribuiti:

- rilevare un *deadlock*, cioè un gruppo di processi bloccati, ciascuno in attesa di un altro;
- capire se una computazione distribuita è terminata;
- accorgersi che il *token* di un protocollo a gettone è andato perso;
- trovare gli oggetti che nessun processo può più raggiungere, per liberarne la memoria (*garbage collection* distribuita);
- salvare uno stato da cui ripartire dopo un guasto (*checkpointing*);
- riconfigurare un programma mentre è in esecuzione, per esempio per bilanciare il carico;
- fare *debugging* e monitoraggio di un'applicazione distribuita.

In tutti questi casi la condizione da controllare riguarda lo **stato globale** del sistema, cioè l'insieme degli stati dei singoli processi, e si può scrivere come un predicato Φ su di esso: «c'è un ciclo di processi in attesa l'uno dell'altro», «tutti i processi sono inattivi e non ci sono messaggi in viaggio», e così via. Il problema astratto si chiama **Global Predicate Evaluation** (GPE, valutazione di un predicato globale): stabilire se lo stato globale del sistema soddisfa Φ . Una soluzione generale di GPE è

il cuore della soluzione di tutti i problemi elencati: per ciascuno resta da scegliere il predicato giusto e la reazione da eseguire quando è vero.

La difficoltà sta nel fatto che i processi non condividono memoria e comunicano solo scambiandosi messaggi. Un processo che vuole conoscere lo stato globale deve farsi mandare gli stati degli altri, e i messaggi impiegano tempi variabili e imprevedibili. Lo stato che ricostruisce può quindi essere:

- *obsoleto*: nel frattempo il sistema è andato avanti;
- *incompleto*: alcune informazioni sono ancora in viaggio;
- *inconsistente*: mette insieme pezzi presi in momenti incompatibili, e descrive una situazione che non si è mai verificata.

Comunicare più spesso rende la vista più aggiornata e più completa, ma non basta a renderla consistente: bisogna ragionare sull'ordine in cui le informazioni vengono osservate e su che cosa contengono. Il capitolo 2 definisce con precisione la consistenza. I capitoli successivi mostrano due modi di costruire stati globali consistenti: in uno un processo *osserva* passivamente il sistema (capitoli 3 e 4), nell'altro lo *interroga* attivamente (capitolo 5).

1.2 Il modello: sistemi distribuiti asincroni

Definizione (sistema distribuito). Un **sistema distribuito** è un insieme di processi sequenziali $p_1, \dots, p_n$ e una rete che realizza **canali** di comunicazione unidirezionali tra coppie di processi. Nel nostro modello:

- i canali sono **affidabili**: ogni messaggio inviato prima o poi arriva;
- i canali **non sono FIFO**: possono consegnare i messaggi in un ordine diverso da quello di invio;
- la rete è **fortemente connessa**: ogni processo può comunicare con ogni altro, eventualmente passando per processi intermedi, anche se non c'è un canale diretto tra ogni coppia.

Quando si sceglie un modello conviene fare meno ipotesi possibile. Le due grandezze su cui si può contare o no sono la velocità relativa dei processi e il ritardo dei messaggi.

Definizione (sistema asincrono). Un sistema distribuito è **asincrono** se non esiste alcun limite né alle velocità relative dei processi né ai ritardi dei messaggi. Se invece questi limiti esistono e sono noti, il sistema è **sincrono**.

In un sistema asincrono un processo non può contare su orologi sincronizzati con quelli degli altri, né ragionare in termini di un tempo reale comune: l'unico strumento di sincronizzazione è la comunicazione. Ci sono due buoni motivi per studiare proprio questo modello.

- *È il più debole*. Fa meno ipotesi di qualunque altro, quindi una soluzione che funziona in un sistema asincrono funziona in qualunque sistema distribuito. Il

suo costo è anche un limite superiore: in ogni altro sistema lo stesso problema si risolve con un costo non maggiore.

- *È realistico*. I componenti fisici sono spesso sincroni (le velocità dei processori e i ritardi sui collegamenti sono limitati), ma gli strati di software che li condividono tra molti processi, come *scheduling*, code e ritrasmissioni, rendono imprevedibili i tempi visti dalle applicazioni. Il sistema che ne risulta si descrive meglio come asincrono.

Non vale invece il contrario. Un algoritmo pensato per un sistema sincrono può sfruttare i limiti di tempo, per esempio concludendo che un processo non ha inviato nulla perché entro un certo tempo non è arrivato niente; in un sistema asincrono questo ragionamento è sbagliato, perché un ritardo lungo è sempre possibile.

Il teorema CAP. Un risultato classico sui sistemi che conservano dati replicati su più nodi è il **teorema CAP**. Riguarda tre proprietà:

- *Consistency* (C): tutti i nodi vedono gli stessi dati, come se ne esistesse una sola copia, e una lettura restituisce sempre l'ultimo valore scritto;
- *Availability* (A): ogni richiesta a un nodo funzionante riceve una risposta;
- *Partition tolerance* (P): il sistema continua a funzionare anche se la rete si divide in parti che non riescono a comunicare tra loro.

Il teorema afferma che un sistema distribuito non può garantirle tutte e tre insieme, ma al massimo due. Poiché in una rete reale le partizioni possono sempre capitare, in pratica la scelta è tra consistenza e disponibilità durante una partizione. Attenzione al nome: la *consistency* di CAP riguarda le copie dei dati e non c'entra con la consistenza di tagli e stati globali del capitolo 2.

1.3 Computazioni distribuite

Una **computazione distribuita** è l'esecuzione di un programma distribuito da parte dei processi del sistema. L'attività di ogni processo, che è sequenziale, si descrive come una sequenza di **eventi**. Un evento può essere:

- **interno**: cambia soltanto lo stato locale del processo;
- un **invio** *send*(*m*): il processo mette il messaggio *m* sul canale verso il destinatario;
- una **ricezione** *receive*(*m*): il processo preleva *m* dal canale in arrivo. Perché avvenga, *m* deve essere arrivato e il processo deve essere pronto a riceverlo; altrimenti uno dei due aspetta l'altro.

Supponiamo che ogni messaggio abbia un identificatore unico (basta aggiungere al contenuto l'indice del mittente e un numero di sequenza): così ogni ricezione corrisponde esattamente a un invio, anche se più processi inviano lo stesso dato. A questo livello di astrazione qualunque forma di comunicazione di più alto livello, come chiamate di procedura remota, *broadcast*, transazioni distribuite o memoria condivisa distribuita, si riduce a coppie di eventi di invio e ricezione.

Definizione (storia locale e storia globale). La **storia locale** di p_i è la sequenza, eventualmente infinita, dei suoi eventi

$$h_i = e_i^1 e_i^2 e_i^3 \dots$$

numerati nell'ordine in cui p_i li esegue: è l'**enumerazione canonica**. Con $h_i^k = e_i^1 \dots e_i^k$ si indica il prefisso dei primi k eventi (h_i^0 è la sequenza vuota). La **storia globale** è l'insieme di tutti gli eventi della computazione: $H = h_1 \cup \dots \cup h_n$.

1.3.1 Precedenza causale

La storia globale dice quali eventi sono avvenuti, ma non *quando* gli eventi di processi diversi sono avvenuti l'uno rispetto all'altro: in un sistema asincrono non c'è un tempo comune a cui riferirsi. L'unico ordine che ha senso è quello di causa ed effetto: un evento deve venire prima di un altro solo se può averlo influenzato, cioè se c'è stato un flusso di informazione dal primo al secondo. L'informazione può passare in due modi: tra due eventi dello stesso processo, che lavorano sullo stesso stato locale, oppure dall'invio alla ricezione di uno stesso messaggio.

Definizione (precedenza causale). La **precedenza causale** $\rightarrow$ (*causal precedence*) è la relazione tra gli eventi di H definita da tre regole:

1. se e ed e' sono eventi dello stesso processo ed e viene prima di e' , allora $e \rightarrow e'$;
2. se $e = \text{send}(m)$ ed $e' = \text{receive}(m)$, allora $e \rightarrow e'$;
3. se $e \rightarrow e'$ ed $e' \rightarrow e''$, allora $e \rightarrow e''$ (transitività).

Due eventi sono **concorrenti**, e si scrive $e \parallel e'$, se non vale né $e \rightarrow e'$ né $e' \rightarrow e$.

$e \rightarrow e'$ si legge « e precede causalmente e' »: l'esecuzione di e , e il suo risultato, *possono* aver influenzato e' . Solo per una coppia invio-ricezione l'influenza è certa; in generale si può dire soltanto che è possibile. Lamport, che ha introdotto la relazione, la chiamava *happened before*; qui si evita quel nome perché suggerisce un ordine nel tempo reale, mentre conta solo la possibilità di un'influenza. Due eventi concorrenti, invece, non possono essersi influenzati in nessun verso, e il loro ordine nel tempo reale è irrilevante.

Una computazione distribuita, quindi, non è una sequenza di eventi ma un insieme parzialmente ordinato (*poset*): formalmente è la coppia $(H, \rightarrow)$. L'ordine è *parziale* perché le coppie di eventi concorrenti non sono ordinate. L'ordine degli eventi di ciascun processo e gli abbinamenti tra invii e ricezioni sono già contenuti in H , perché gli eventi sono numerati e i messaggi hanno identificatori unici.

1.3.2 Diagrammi spazio-tempo

Una computazione si rappresenta comodamente con un **diagramma spazio-tempo** (*space-time diagram*) come quello della Figura 1.1. Ogni processo è una linea

orizzontale, con il tempo che scorre da sinistra a destra; gli eventi sono punti sulla linea, e ogni messaggio è una freccia che parte dall'evento di invio e arriva all'evento di ricezione. Gli eventi interni non hanno frecce.

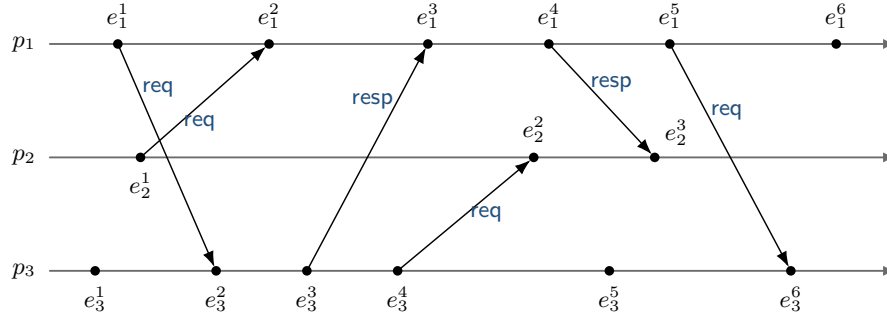


Figura 1.1. La computazione d'esempio: tre processi che si scambiano richieste (req) e risposte (resp).

Esempio (la computazione d'esempio). La computazione della Figura 1.1 tornerà in quasi tutti i capitoli. È un sistema client-server con chiamate di procedura remota: ogni processo può fare sia da client sia da server, e dopo aver inviato una richiesta (req) aspetta la risposta (resp). Per esempio p_1 invia una richiesta a p_3 (e_1^1); p_3 la riceve (e_3^2) e risponde (e_3^3), e la risposta arriva a p_1 in e_1^3 . Nel frattempo p_1 ha ricevuto una richiesta da p_2 (e_2^1), a cui risponde in e_1^4 . Gli eventi e_3^1 , e_3^5 e e_1^6 sono interni.

Il diagramma rende facile leggere la precedenza causale: $e \rightarrow e'$ se e solo se si può andare da e a e' con un cammino che si muove verso destra lungo le linee dei processi e segue le frecce nel loro verso. Se un cammino così non c'è in nessuno dei due versi, i due eventi sono concorrenti.

Esempio (leggere la precedenza causale). Nella Figura 1.1:

- $e_2^1 \rightarrow e_3^6$, grazie al cammino $e_2^1 \rightarrow e_1^2 \rightarrow e_1^3 \rightarrow e_1^4 \rightarrow e_1^5 \rightarrow e_3^6$: la richiesta di p_2 arriva a p_1 , si prosegue lungo p_1 , e da e_1^5 parte la richiesta che p_3 riceve in e_3^6 ;
- $e_2^2 \parallel e_3^6$: dopo e_2^2 il processo p_2 non invia più messaggi, quindi da e_2^2 non si arriva a p_3 ; e e_3^6 è l'ultimo evento di p_3 , quindi da lì non si arriva da nessuna parte;
- $e_1^1 \parallel e_2^1$: sono i primi eventi dei rispettivi processi e nessun messaggio li collega.

Capitolo 2

Stati globali, tagli e consistenza

Per valutare un predicato globale bisogna prima ricostruire uno stato globale. Questo capitolo definisce stati locali e globali e i tagli che li individuano, e mostra con un esempio come un monitor ingenuo possa ricostruire uno stato mai esistito e trarne conclusioni sbagliate. La nozione che separa gli stati sensati da quelli impossibili è la consistenza; gli stati consistenti, ordinati, formano il reticolo degli stati globali.

2.1 Stati globali, tagli e run

Indichiamo con σ_i^k lo **stato locale** di p_i subito dopo l'evento e_i^k , e con σ_i^0 il suo stato iniziale, prima di qualunque evento. Lo stato locale comprende i valori delle variabili del processo e, se servono, le sequenze dei messaggi inviati e ricevuti su ciascun canale. Lo **stato globale** è la n -upla degli stati locali, uno per processo: $\Sigma = (\sigma_1, \dots, \sigma_n)$.

Non serve aggiungere allo stato globale lo stato dei canali, cioè i messaggi in viaggio: se gli stati locali registrano i messaggi inviati e ricevuti, i messaggi in viaggio da p_i a p_j sono quelli che p_i ha inviato e p_j non ha ancora ricevuto. Gli stati dei canali torneranno in modo esplicito con gli snapshot (capitolo 5).

Per indicare *quale* stato globale si considera basta dire fino a che punto è arrivato ciascun processo. È l'idea di taglio.

Definizione (taglio). Un **taglio** (*cut*) è un sottoinsieme C della storia globale formato da un prefisso della storia locale di ogni processo:

$$C = h_1^{c_1} \cup \dots \cup h_n^{c_n}.$$

Si indica con la tupla $(c_1, \dots, c_n)$ degli indici dell'ultimo evento incluso per ciascun processo. Gli ultimi eventi inclusi, $(e_1^{c_1}, \dots, e_n^{c_n})$, formano la **frontiera** del taglio, e lo stato globale che gli corrisponde è $(\sigma_1^{c_1}, \dots, \sigma_n^{c_n})$.

Nel diagramma spazio-tempo un taglio è una linea che attraversa dall'alto in basso tutti i processi: gli eventi a sinistra della linea sono nel taglio, quelli a destra no. La Figura 2.1 mostra due tagli della computazione d'esempio, $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

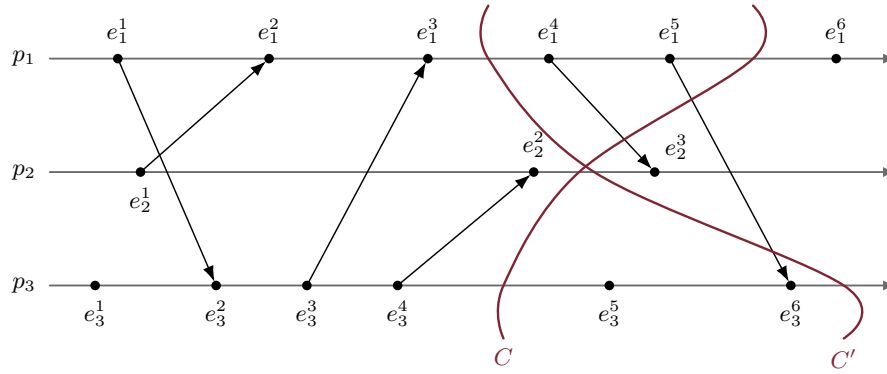


Figura 2.1. Due tagli della computazione d'esempio: $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

Un taglio dice quali eventi sono avvenuti, non in che ordine. Per ragionare sulle esecuzioni serve anche un ordine totale: in un'esecuzione reale tutti gli eventi, anche di processi diversi, avvengono in un qualche ordine. Due eventi simultanei si possono ordinare in modo arbitrario, per esempio mettendo prima quello del processo con indice minore.

Definizione (run). Un **run** (esecuzione) di una computazione è un ordine totale di tutti gli eventi della storia globale che rispetta ogni storia locale: gli eventi di ciascun processo p_i compaiono nel run nello stesso ordine in cui compaiono in h_i .

Una computazione ha in generale molti run, e un run non corrisponde per forza a un'esecuzione possibile: la definizione non impedisce, per esempio, di mettere la ricezione di un messaggio prima del suo invio. A questo rimediano i run consistenti della Sezione 2.3.

2.2 Monitoraggio attivo e deadlock fantasma

Supponiamo che un solo processo, il **monitor** p_0 , abbia il compito di valutare il predicato Φ . Può essere uno dei processi $p_1, \dots, p_n$ oppure un processo esterno alla computazione, ma pur sempre parte del sistema e soggetto ai suoi ritardi. Gli eventi eseguiti per il monitoraggio non fanno parte della computazione e non cambiano la numerazione dei suoi eventi. Con un solo monitor, risolvere GPE significa costruire uno stato globale a cui applicare Φ .

La strategia più semplice è attiva: p_0 chiede lo stato a tutti. Invia a ogni processo una richiesta; quando la riceve, p_i risponde con il proprio stato locale corrente; raccolte tutte le risposte, p_0 compone lo stato globale. I punti in cui le richieste arrivano ai processi definiscono un taglio, e lo stato costruito è quello di quel taglio. Il problema è che le richieste arrivano in momenti scollegati tra loro, quindi il taglio che ne risulta può essere qualunque.

Esempio (deadlock fantasma). Nella computazione d'esempio (Figura 1.1) ogni processo, dopo aver inviato una richiesta, resta bloccato finché non riceve la risposta, quindi il sistema può andare in deadlock. Per rilevarlo, ogni server tiene nel proprio stato locale l'elenco dei client da cui ha ricevuto richieste a cui non ha ancora risposto. Con queste informazioni si costruisce il **grafo delle attese** (*waits-for graph*, WFG): i nodi sono i processi, e c'è un arco da p_i a p_j se p_j ha ricevuto una richiesta da p_i e non ha ancora risposto, cioè se p_i sta aspettando p_j . Un ciclo nel grafo è un deadlock, perché ogni processo del ciclo aspetta il successivo. Il predicato da valutare è quindi $\Phi = \ll \text{il WFG contiene un ciclo} \gg$.

Supponiamo che le richieste di p_0 arrivino ai processi nei punti del taglio $C' = (3, 2, 6)$ della Figura 2.1, e che p_0 riceva quindi σ_1^3 , σ_2^2 e σ_3^6 :

- in σ_3^6 , p_3 ha ricevuto da p_1 la richiesta inviata in e_1^5 e non ha risposto: arco $p_1 \rightarrow p_3$;
- in σ_1^3 , p_1 ha ricevuto la richiesta di p_2 (e_1^2) e non ha ancora risposto: arco $p_2 \rightarrow p_1$;
- in σ_2^2 , p_2 ha ricevuto la richiesta di p_3 (e_2^2) e non ha risposto: arco $p_3 \rightarrow p_2$.

Il grafo contiene il ciclo $p_1 \rightarrow p_3 \rightarrow p_2 \rightarrow p_1$ (Figura 2.2) e p_0 dichiara un deadlock. Ma un osservatore esterno che vedesse l'intera computazione direbbe che il sistema non è mai in deadlock: p_1 risponde a p_2 (e_1^4) *prima* di inviare la nuova richiesta a p_3 (e_1^5). Il deadlock rilevato è un **deadlock fantasma** (*ghost deadlock*).



Figura 2.2. Il grafo delle attese costruito sullo stato del taglio C e su quello di C' : solo il secondo contiene un ciclo.

Dov'è l'errore? Nello stato di C' , p_3 risulta aver ricevuto una richiesta (e_3^6) che p_1 non ha ancora inviato (e_1^5 è fuori dal taglio): uno stato così non si è mai verificato, e non avrebbe mai potuto verificarsi. Se invece le richieste di p_0 arrivassero nei punti del taglio $C = (5, 2, 4)$, il grafo avrebbe il solo arco $p_3 \rightarrow p_2$ e nessun ciclo. Ogni taglio corrisponde a uno stato globale, ma solo alcuni corrispondono a stati in cui il sistema si sarebbe potuto trovare davvero.

2.3 Tagli e run consistenti

La precedenza causale è lo strumento giusto per distinguere i due tipi di taglio.

Definizione (taglio consistente). Un taglio C è **consistente** se, per ogni coppia di eventi e ed e' ,

$$(e \in C) \wedge (e' \rightarrow e) \Rightarrow e' \in C.$$

In parole: se un evento sta nel taglio, ci stanno anche tutti gli eventi che lo precedono causalmente. Un taglio consistente è «chiuso all'indietro» rispetto alla precedenza causale: non contiene un effetto senza le sue cause.

Dentro un singolo processo questa condizione è sempre soddisfatta, perché un taglio contiene prefissi delle storie locali. L'unico modo per violarla è un messaggio *ricevuto* dentro il taglio ma *inviato* fuori. Nel diagramma il test è immediato: un taglio è consistente se ogni freccia che lo attraversa va da sinistra (dentro il taglio) verso destra (fuori); è inconsistente se una freccia lo attraversa da destra verso sinistra, cioè se c'è un messaggio ricevuto ma non ancora inviato. Nella Figura 2.1 il taglio C è consistente; C' no, perché la richiesta da e_1^5 a e_3^6 lo attraversa all'indietro.

Uno **stato globale consistente** è lo stato di un taglio consistente. Sono esattamente gli stati che un osservatore ideale, esterno al sistema, avrebbe potuto vedere durante un'esecuzione. Il deadlock fantasma nasce dall'aver valutato Φ in uno stato inconsistente: il valore di un predicato ha senso solo negli stati globali consistenti.

I tagli consistenti hanno un ruolo centrale. In un programma sequenziale un istante di tempo individua un punto preciso dell'esecuzione; in una computazione distribuita questo ruolo lo ha la frontiera di un taglio consistente. Anche «prima» e «dopo» vanno intesi rispetto ai tagli consistenti: un evento è prima di un taglio se sta a sinistra della sua frontiera, dopo se sta a destra.

La stessa idea si applica ai run.

Definizione (run consistente). Un run R è **consistente** se, per ogni coppia di eventi, $e \rightarrow e'$ implica che e compare prima di e' in R .

In altre parole, l'ordine totale del run estende l'ordine parziale della precedenza causale. Un run $R = e^1 e^2 \dots$ si può vedere anche come una sequenza di stati globali $\Sigma^0 \Sigma^1 \Sigma^2 \dots$, dove $\Sigma^0 = (\sigma_1^0, \dots, \sigma_n^0)$ è lo stato iniziale e ogni Σ^i si ottiene da Σ^{i-1} eseguendo il singolo evento e^i . Se il run è consistente, ogni suo prefisso è un taglio consistente, quindi tutti gli stati della sequenza sono consistenti.

Due stati consecutivi di un run sono legati dalla relazione *leads to* («porta a»): Σ^{i-1} porta a Σ^i nel run R . La sua chiusura transitiva si scrive $\rightsquigarrow_R$, e Σ' è **raggiungibile** da Σ nel run R se $\Sigma \rightsquigarrow_R \Sigma'$. Si scrive semplicemente $\Sigma \rightsquigarrow \Sigma'$ se esiste *qualche* run in cui Σ' è raggiungibile da Σ .

2.4 Il reticolo degli stati globali

L'insieme degli stati globali consistenti di una computazione, con la relazione *leads to*, forma un **reticolo** (*lattice*). Con n processi il reticolo ha n assi, uno per processo. Si abbrevia con $\Sigma^{k_1 \dots k_n}$ lo stato globale $(\sigma_1^{k_1}, \dots, \sigma_n^{k_n})$, e si chiama **livello** la somma $k_1 + \dots + k_n$, cioè il numero di eventi eseguiti.

La Figura 2.3 mostra una computazione di due processi e il suo reticolo. Ogni passo verso il basso esegue un evento: scendendo a sinistra avanza p_1 , scendendo a destra avanza p_2 . Ogni stato è raggiungibile dallo stato iniziale Σ^{00} , e gli stati inconsistenti mancano. Manca per esempio Σ^{20} : in e_1^2 il processo p_1 riceve il messaggio inviato da p_2 in e_2^1 , quindi p_1 non può aver eseguito due eventi se p_2 non ne ha eseguito nessuno. Per lo stesso motivo mancano gli stati in cui p_1 ha eseguito e_1^5 ma p_2 non ha ancora eseguito e_2^3 , e quelli in cui p_2 ha eseguito e_2^5 ma p_1 non ha ancora eseguito e_1^3 .

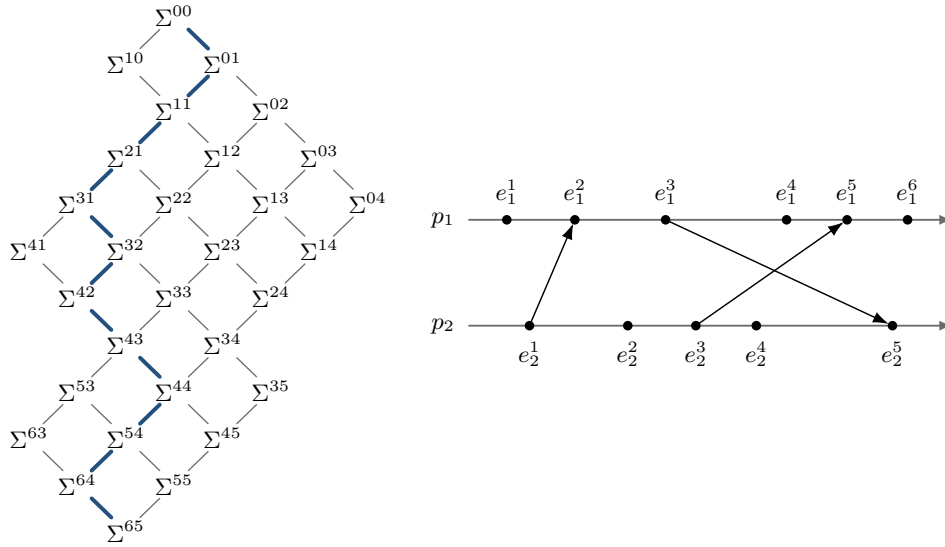


Figura 2.3. Una computazione di due processi (a destra) e il reticolo dei suoi stati globali consistenti (a sinistra). In blu un cammino, cioè un run consistente.

Un **cammino** nel reticolo è una sequenza di stati in cui ogni stato ha un evento in più del precedente. Ogni cammino dallo stato iniziale a quello finale corrisponde a un run consistente, e ogni run consistente è un cammino. Il run evidenziato in blu nella figura, per esempio, passa per gli stati

$$\Sigma^{00} \Sigma^{01} \Sigma^{11} \Sigma^{21} \Sigma^{31} \Sigma^{32} \Sigma^{42} \Sigma^{43} \Sigma^{44} \Sigma^{54} \Sigma^{64} \Sigma^{65}.$$

Si sarebbe tentati di cercare quale di questi cammini sia l'esecuzione *effettiva*. In un sistema asincrono, però, dall'interno del sistema è impossibile saperlo: solo un osservatore esterno onnisciente conoscerebbe gli stati per cui l'esecuzione è passata davvero. Per chi sta dentro il sistema, tutti i cammini sono ugualmente plausibili.

Tre proprietà dei tagli consistenti aiutano a ragionare sul reticolo.

- *Intersezione e unione di due tagli consistenti sono tagli consistenti.* Se un evento sta in entrambi i tagli, le sue cause stanno in entrambi; se sta in almeno uno dei due, le sue cause stanno almeno in quello. È questo a fare dell'insieme un reticolo: per due stati consistenti esistono sempre il più grande stato da cui entrambi sono raggiungibili (quello dell'intersezione) e il più piccolo stato raggiungibile da entrambi (quello dell'unione).
- *Ogni stato globale consistente è raggiungibile dallo stato iniziale con un run consistente.* Basta eseguire gli eventi del taglio uno alla volta, scegliendo ogni

volta un evento le cui cause sono già state eseguite. Un evento così esiste sempre, perché la precedenza causale non ha cicli.

- *Se $C \subseteq C'$ sono due tagli consistenti, lo stato di C' è raggiungibile da quello di C .* Si prende un run consistente che arriva allo stato di C e lo si prolunga con gli eventi di $C' \setminus C$, sempre in un ordine che rispetta la precedenza causale.

L'ultima proprietà tornerà utile con gli snapshot (Sezione 5.6).

Capitolo 3

Osservare una computazione: orologio reale e clock logici

Nel capitolo precedente il monitor interrogava attivamente i processi e otteneva un taglio qualunque. Qui si prova la strategia opposta: il monitor non chiede nulla, sono i processi a informarlo di ogni evento, e il suo compito diventa mettere in ordine le notifiche in modo che la sequenza osservata sia consistente. Si parte da ipotesi forti, un orologio globale e ritardi limitati, e le si toglie una alla volta. Lo strumento centrale del capitolo sono i clock logici di Lamport.

3.1 Monitoraggio passivo e osservazioni

Nella strategia passiva il monitor p_0 non invia messaggi. I processi della computazione vengono modificati di poco: ogni volta che eseguono un evento, lo notificano a p_0 con un messaggio che lo descrive. L'invio della notifica non è un evento nuovo, ma coincide con l'evento notificato. In pratica basta notificare gli eventi **rilevanti** per Φ , cioè quelli che possono cambiarne il valore: per il deadlock della computazione d'esempio sono gli invii e le ricezioni di richieste e risposte, gli unici che modificano il grafo delle attese. Il monitor costruisce così un'**osservazione** della computazione: la sequenza degli eventi nell'ordine in cui arrivano le notifiche.

Le osservazioni hanno due caratteristiche importanti.

- Poiché i ritardi variano, la stessa esecuzione può produrre osservazioni diverse su monitor diversi. È il cosiddetto **effetto relativistico** del calcolo distribuito: non esiste un punto di vista privilegiato.
- Un'osservazione può corrispondere a un run consistente, a un run inconsistente o a nessun run, se le notifiche di uno stesso processo arrivano in disordine. Si dice **consistente** un'osservazione che corrisponde a un run consistente.

Esempio (tre osservazioni della stessa esecuzione). Supponiamo che la computazione d'esempio (Figura 1.1) venga notificata a p_0 . Con canali non FIFO qualunque ordine di arrivo delle notifiche è possibile.

- Se la notifica di e_3^4 arriva prima di quella di e_3^3 , la sequenza osservata non

rispetta nemmeno la storia locale di p_3 : non è un run.

- Se le notifiche di e_1^4 ed e_1^5 tardano mentre quella di e_3^6 arriva presto, p_0 vede p_3 ricevere una richiesta che p_1 non ha ancora inviato: l'osservazione è un run inconsistente. Se a quel punto p_0 valuta Φ , si trova proprio nello stato del taglio C' , cioè $(\sigma_1^3, \sigma_2^2, \sigma_3^6)$, e ritrova il deadlock fantasma.
- Se le notifiche arrivano in un ordine che rispetta la precedenza causale, l'osservazione è consistente, e tutti gli stati che p_0 attraversa sono consistenti: per esempio quello del taglio C .

3.2 Regole di consegna e consegna FIFO

Per rimettere in ordine i messaggi, chi li riceve non li passa subito all'applicazione, ma decide quando farlo seguendo una **regola di consegna** (*delivery rule*). Si distinguono quindi due operazioni: con *receive* il messaggio arriva dal canale, con *deliver* viene consegnato all'applicazione. La ricezione resta nascosta dentro la regola di consegna e non compare nella storia locale del processo: per l'applicazione conta solo la consegna.

Definizione (consegna FIFO). La comunicazione da p_i a p_j rispetta la **consegna FIFO** (*First In, First Out*) se, per ogni coppia di messaggi m ed m' ,

$$send_i(m) \rightarrow send_i(m') \Rightarrow deliver_j(m) \rightarrow deliver_j(m'),$$

dove il pedice indica il processo che esegue l'evento.

In parole: un messaggio non può sorpassare un messaggio inviato prima dallo stesso mittente. La consegna FIFO si realizza facilmente anche su canali che non la garantiscono: il mittente numera i messaggi che invia a ciascun destinatario, e il destinatario li consegna in ordine di numero, tenendo da parte quelli arrivati in anticipo. Se per esempio arrivano prima i messaggi numero 2 e 3, il destinatario li trattiene finché non arriva l'1, e poi consegna 1, 2 e 3.

Con canali FIFO tra i processi e il monitor, le notifiche di ogni processo arrivano nell'ordine giusto e ogni osservazione è un run: il primo caso dell'esempio non può più capitare. La consistenza però non è garantita: il secondo caso riguarda notifiche di processi *diversi*, p_1 e p_3 , che la consegna FIFO non ordina tra loro. Serve un meccanismo che ordini le notifiche secondo la precedenza causale.

3.3 Orologio reale e clock condition

Partiamo da ipotesi forti, che poi verranno tolte. Supponiamo che tutti i processi possano leggere un orologio reale globale e che ogni messaggio arrivi entro un ritardo massimo noto δ . Non è più un sistema asincrono, ma è un buon punto di partenza. Sia $RC(e)$ il valore dell'orologio quando viene eseguito l'evento e : ogni notifica porta con sé come **timestamp** (marca temporale) il valore $RC(e)$ dell'evento notificato, e il monitor segue questa regola.

Definizione (regola DR1). Al tempo t , consegna tutti i messaggi ricevuti con timestamp fino a $t - \delta$, in ordine di timestamp crescente.

Perché funziona? Un messaggio con timestamp non superiore a $t - \delta$ è stato inviato almeno δ prima di t , quindi al tempo t è sicuramente già arrivato. Quando il monitor consegna i messaggi con timestamp fino a $t - \delta$, perciò, non può più arrivare un messaggio con timestamp minore di quelli già consegnati. L'osservazione ordina quindi gli eventi per timestamp crescente: e viene osservato prima di e' se e solo se $RC(e) < RC(e')$ (a parità di timestamp decide l'indice del processo). Allora l'osservazione è consistente se e solo se i timestamp rispettano la precedenza causale.

Definizione (clock condition). Una funzione T che associa un valore a ogni evento soddisfa la **clock condition** se

$$e \rightarrow e' \Rightarrow T(e) < T(e').$$

L'orologio reale la soddisfa senza fatica, perché una causa avviene sempre prima del suo effetto. L'osservazione chiave è che la clock condition è l'*unica* proprietà dell'orologio reale che serve: qualunque meccanismo di timestamp che la soddisfi si può usare al posto del tempo reale. E un meccanismo così esiste anche in un sistema asincrono.

Attenzione al verso dell'implicazione: la clock condition dice che le cause hanno timestamp più piccoli degli effetti, non che un timestamp più piccolo indichi una causa. Due eventi con $T(e) < T(e')$ possono essere legati causalmente oppure concorrenti; l'unica cosa certa è che e' non precede causalmente e .

3.4 Clock logici di Lamport

Il clock logico di Lamport è un contatore che ogni processo mantiene per conto proprio e che fa da orologio senza alcuna sincronizzazione tra i processi.

Definizione (clock logico di Lamport). Ogni processo p_i mantiene un contatore LC , il suo **clock logico** (*logical clock*), inizialmente 0. Ogni messaggio m porta un timestamp $TS(m)$, il valore del clock del mittente nell'evento di invio. A ogni nuovo evento e_i , p_i aggiorna il clock così:

$$LC(e_i) = \begin{cases} LC + 1 & \text{se } e_i \text{ è un evento interno o un invio,} \\ \max\{LC, TS(m)\} + 1 & \text{se } e_i = \text{receive}(m). \end{cases}$$

Un evento interno o un invio fa avanzare il clock di uno. Una ricezione porta il clock oltre sia il proprio valore precedente sia il timestamp del messaggio: il destinatario salta in avanti per non restare indietro rispetto al passato del mittente.

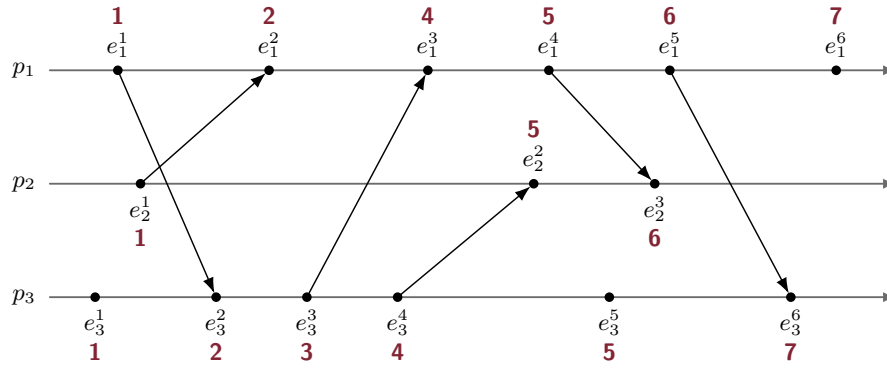


Figura 3.1. I clock logici della computazione d'esempio: accanto a ogni evento, in rosso, il valore di LC .

Esempio (clock logici passo passo). Applichiamo le regole alla computazione d'esempio (Figura 3.1). All'inizio tutti i clock valgono 0.

- I primi eventi e_1^1 , e_2^1 ed e_3^1 sono invii o eventi interni, e il loro clock vale 1. Le richieste inviate in e_1^1 ed e_2^1 portano timestamp 1.
- e_1^2 riceve la richiesta di p_2 , con timestamp 1: $\max\{1, 1\} + 1 = 2$. Allo stesso modo e_3^2 , che riceve la richiesta di p_1 , vale 2.
- p_3 invia la risposta, $LC(e_3^3) = 3$, e poi una richiesta a p_2 , $LC(e_3^4) = 4$.
- e_1^3 riceve la risposta di p_3 , con timestamp 3: $\max\{2, 3\} + 1 = 4$. Il clock di p_1 salta da 2 a 4 per superare il passato di p_3 .
- e_2^2 riceve la richiesta di p_3 , con timestamp 4: $\max\{1, 4\} + 1 = 5$.
- p_1 prosegue con la risposta a p_2 , $LC(e_1^4) = 5$, la richiesta a p_3 , $LC(e_1^5) = 6$, e l'evento interno e_1^6 , che vale 7.
- e_2^3 riceve la risposta di p_1 , con timestamp 5: $\max\{5, 5\} + 1 = 6$.
- e_3^5 è interno e vale 5; e_3^6 riceve la richiesta inviata in e_1^5 , con timestamp 6: $\max\{5, 6\} + 1 = 7$. Il clock di p_3 passa da 5 a 7 e non assume mai il valore 6.

I clock logici crescono lungo ogni catena causale: a ogni passo dentro un processo il clock aumenta di uno, e a ogni messaggio il clock del destinatario supera il timestamp del mittente. Ne segue la proprietà fondamentale.

I clock logici soddisfano la clock condition. Per ogni coppia di eventi

$$e \rightarrow e' \Rightarrow LC(e) < LC(e').$$

Il viceversa non vale, e dai soli valori dei clock non si capisce se due eventi siano legati o concorrenti. Nella Figura 3.1, $LC(e_2^2) = 5 < LC(e_3^6) = 7$, ma i due eventi sono concorrenti, come si è visto nel capitolo 1. Da $LC(e) < LC(e')$ si deduce

soltanto che e' non precede causalmente e .

Ordinando tutti gli eventi per clock logico crescente, e risolvendo le parità con l'indice del processo, si ottiene un ordine totale che rispetta la precedenza causale, cioè un run consistente. Per la computazione d'esempio è

$$e_1^1 e_2^1 e_3^1 e_1^2 e_3^2 e_3^3 e_1^4 e_3^4 e_1^2 e_3^5 e_1^5 e_2^3 e_1^6 e_3^6.$$

3.5 Consegnare le notifiche in ordine di clock logico

Torniamo al monitor. Se i processi usano i clock logici come timestamp delle notifiche e p_0 le consegna in ordine di timestamp crescente, l'osservazione è consistente, perché i clock logici soddisfano la clock condition. Resta però un problema serio: senza un limite ai ritardi, p_0 non può mai essere sicuro che più tardi non arrivi una notifica con timestamp minore di quelle che ha in mano. Per prudenza non consegnerebbe mai nulla: la regola non ha *liveness*, cioè non garantisce che prima o poi qualcosa venga consegnato. Manca la capacità di accorgersi dei «buchi».

Definizione (gap detection). Un meccanismo di clock ha la proprietà di **gap detection** se, dati due eventi e ed e' con $LC(e) < LC(e')$, permette di stabilire se esiste un altro evento e'' con $LC(e) < LC(e'') < LC(e')$.

I clock logici non hanno questa proprietà: i valori possono saltare, come il clock di p_1 che passa da 2 a 4, e guardando i soli timestamp non si sa se da qualche parte ci sia un evento con valore 3 di cui non è ancora arrivata la notifica. Nemmeno l'orologio reale ha la gap detection, ma con il limite δ sui ritardi il controllo diventa facile: al tempo t sono arrivati tutti i messaggi con timestamp minore di $t - \delta$.

Con i clock logici serve un'informazione in più, e la fornisce la consegna FIFO. Supponiamo che i canali da ogni processo a p_0 siano FIFO. Poiché il clock di ogni processo cresce e i messaggi di uno stesso mittente arrivano in ordine, quando p_0 riceve da p_i un messaggio con un certo timestamp sa che da p_i non arriveranno più messaggi con timestamp minore o uguale. Un messaggio m ricevuto da p_0 si dice **stabile** se non può più arrivare un messaggio con timestamp minore di $TS(m)$. Con canali FIFO, m è stabile non appena p_0 ha ricevuto da *ogni* altro processo almeno un messaggio con timestamp maggiore di $TS(m)$.

Definizione (regola DR2). Consegna tutti i messaggi ricevuti che sono stabili per p_0 , in ordine di timestamp crescente.

La regola si può ancora bloccare se un processo smette di inviare notifiche, perché p_0 non riceverebbe mai da lui un timestamp più grande. Si rimedia facendo chiedere a p_0 , periodicamente, una conferma a tutti i processi: le risposte hanno timestamp più grandi e rendono stabili i messaggi precedenti.

DR2 funziona, ma è prudente: consegna in ordine di timestamp anche le notifiche di eventi concorrenti, che si potrebbero consegnare in qualunque ordine. Il capitolo 4 mostra come fare di meglio con un clock più informativo.

Capitolo 4

Consegna causale e vector clock

I clock logici permettono di costruire osservazioni consistenti, ma a un prezzo: il monitor consegna le notifiche in ordine di timestamp, e così aspetta anche quando non serve, perché un clock logico non dice se due eventi sono legati causalmente o concorrenti. Questo capitolo introduce la consegna causale, che è esattamente la proprietà di cui ha bisogno il monitor, e i *vector clock* (clock vettoriali), che catturano la precedenza causale in modo esatto e permettono di realizzarla in modo efficiente.

4.1 Consegna causale

La consegna FIFO ordina i messaggi di uno stesso mittente. La consegna causale estende l'ordine a tutti i messaggi legati causalmente, anche se inviati da processi diversi.

Definizione (consegna causale). Una regola di consegna rispetta la **consegna causale** (*causal delivery*, CD) se, per ogni coppia di messaggi m ed m' inviati da p_i e da p_j allo stesso destinatario p_k ,

$$send_i(m) \rightarrow send_j(m') \Rightarrow deliver_k(m) \rightarrow deliver_k(m').$$

La consegna FIFO è il caso particolare $i = j$. Detto in un altro modo: con la consegna causale un processo non può venire a sapere dell'esistenza di un messaggio, attraverso altri messaggi, prima che quel messaggio gli sia stato consegnato.

Avere la consegna FIFO tra ogni coppia di processi non basta. Nella Figura 4.1, p_1 invia m a p_3 e poi un messaggio a p_2 ; p_2 , dopo averlo ricevuto, invia m' a p_3 . L'invio di m precede causalmente quello di m' , ma m' arriva a p_3 per primo. La consegna FIFO è rispettata, perché su ogni canale viaggia un solo messaggio; quella causale no, se p_3 consegna m' prima di m .

Per il monitor la consegna causale è esattamente ciò che serve. La notifica di un evento parte proprio in quell'evento, quindi se $e \rightarrow e'$ anche l'invio della notifica di e precede causalmente quello della notifica di e' . Se p_0 consegna le notifiche rispettando la consegna causale, osserva sempre le cause prima degli effetti, e tutte le sue osservazioni sono consistenti. Le regole DR1 e DR2 del capitolo precedente sono

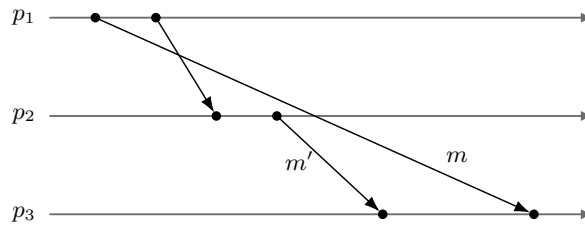


Figura 4.1. Consegna FIFO ma non causale: l'invio di m precede causalmente quello di m' , ma p_3 riceve m' per primo.

modi di realizzare la consegna causale sotto ipotesi particolari: un orologio globale con ritardi limitati, oppure canali FIFO e clock logici. Ora la si vuole ottenere con le sole ipotesi di un sistema asincrono, e senza attese inutili.

4.2 La strong clock condition

DR1 e DR2 consegnano le notifiche in ordine di timestamp strettamente crescente, come se un timestamp più piccolo indicasse una causa. È un'ipotesi prudente: dalla clock condition si sa solo che, se $LC(e) < LC(e')$, allora e' non precede e , ma i due eventi possono essere legati oppure concorrenti. Così, appena arrivata la notifica di e' , il monitor può dover aspettare notifiche con timestamp più piccolo anche quando riguardano eventi concorrenti con e' , che si potrebbero consegnare dopo senza alcun danno. Servirebbe un timestamp da cui si capisca *esattamente* quando due eventi sono legati causalmente.

Definizione (strong clock condition). Un meccanismo di timestamp TC soddisfa la **strong clock condition** se

$$e \rightarrow e' \iff TC(e) < TC(e').$$

Rispetto alla clock condition si aggiunge l'implicazione inversa: un timestamp più piccolo *significa* precedenza causale. Un meccanismo così **caratterizza** la precedenza causale: da una sola osservazione con questi timestamp si ricostruisce l'intera computazione, quindi l'intero reticolo degli stati globali e non solo un suo cammino. Serve per realizzare la consegna causale in modo efficiente, ma anche altrove, per esempio nel *debugging* distribuito, dove interessano tutte le esecuzioni possibili e non una sola.

4.3 Storie causali

Un modo «a forza bruta» per soddisfare la strong clock condition è usare come timestamp di un evento l'insieme di tutti gli eventi che lo precedono causalmente.

Definizione (storia causale). La **storia causale** (*causal history*) di un evento e è l'insieme

$$\theta(e) = \{e' \in H \mid e' \rightarrow e\} \cup \{e\}.$$

La storia causale di e è il più piccolo taglio consistente che contiene e . Nella Figura 4.2 è evidenziata quella di e_1^4 :

$$\theta(e_1^4) = \{e_1^1, e_1^2, e_1^3, e_1^4, e_2^1, e_3^1, e_2^2, e_3^3\}.$$

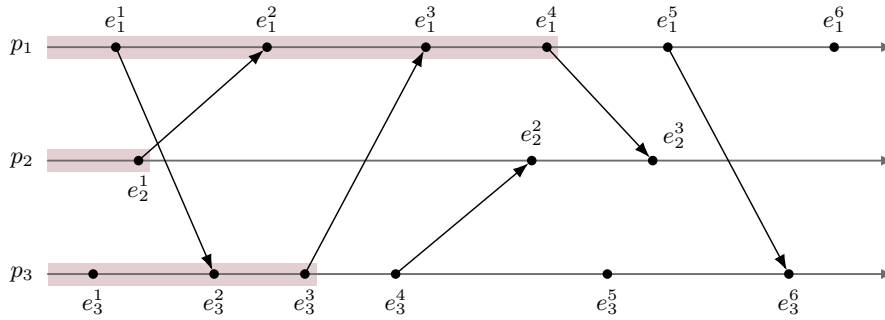


Figura 4.2. La storia causale di e_1^4 : i tratti evidenziati delle storie locali.

Mantenere le storie causali è semplice. Ogni processo parte dall'insieme vuoto. Per un evento interno o un invio, la storia causale è quella dell'evento precedente dello stesso processo più l'evento stesso; per una ricezione si aggiunge anche la storia causale dell'invio corrispondente, che viaggia nel messaggio come timestamp. Il confronto tra timestamp diventa l'inclusione tra insiemi,

$$e \rightarrow e' \iff \theta(e) \subset \theta(e'),$$

e per $e \neq e'$ basta controllare se $e \in \theta(e')$. La strong clock condition è soddisfatta, ma c'è un difetto che rende le storie causali inutilizzabili in pratica: crescono senza limite, e ogni messaggio dovrebbe trasportare insiemi sempre più grandi.

4.4 Vector clock

L'osservazione decisiva è che la parte della storia causale che sta su un processo p_i , $\theta_i(e) = \theta(e) \cap h_i$, è sempre un *prefisso* della storia locale di p_i : se la storia causale contiene e_i^k , contiene anche tutti gli eventi precedenti di p_i , che precedono e_i^k . Un prefisso è individuato dalla sua lunghezza, quindi basta un numero per processo, e la storia causale si può rappresentare con un vettore di n interi.

Definizione (vector clock). Il **vector clock** di un evento e è il vettore $VC(e)$ di n naturali la cui componente i conta gli eventi di p_i nella storia causale di e : $VC(e)[i] = k$ se e solo se $\theta_i(e) = h_i^k$. Si calcola così: ogni processo p_i mantiene un vettore VC , inizialmente tutto a zero, e ogni messaggio m porta come timestamp $TS(m)$ il vector clock dell'evento di invio. A ogni nuovo evento

e_i :

- se e_i è un evento interno o un invio, $VC[i] := VC[i] + 1$;
- se $e_i = receive(m)$, prima $VC := \max\{VC, TS(m)\}$, componente per componente, e poi $VC[i] := VC[i] + 1$.

In parole: un evento interno o un invio aggiunge un evento di p_i al passato conosciuto; una ricezione unisce ciò che sapeva il destinatario con ciò che sapeva il mittente, e poi conta la ricezione stessa. Attenzione: alla ricezione si incrementa solo la componente del processo che riceve, non tutte. Per esempio, dalla storia causale di prima si ottiene $VC(e_1^4) = (4, 1, 3)$: quattro eventi di p_1 , uno di p_2 e tre di p_3 .

Le componenti hanno un significato preciso. Per un evento e_i del processo p_i :

- $VC(e_i)[i]$ è la posizione di e_i nella storia di p_i , cioè il numero di eventi che p_i ha eseguito fino a e_i compreso;
- per $j \neq i$, $VC(e_i)[j]$ è il numero di eventi di p_j che precedono causalmente e_i : quanto p_i «sa» di p_j al momento di e_i .

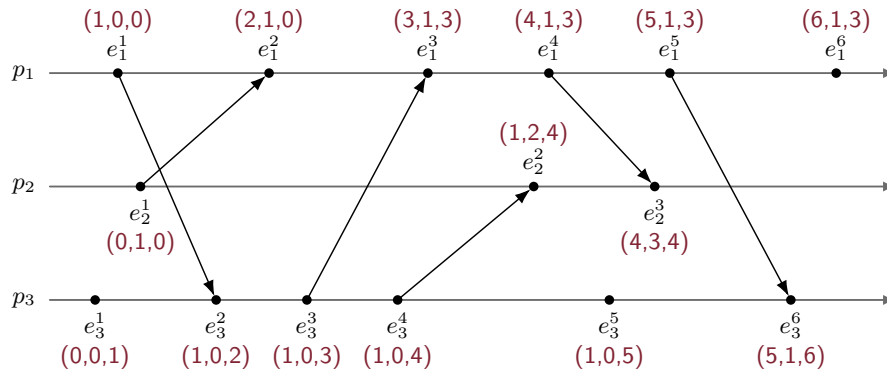


Figura 4.3. I vector clock della computazione d'esempio: accanto a ogni evento, in rosso, il suo vector clock.

Esempio (vector clock passo passo). Applichiamo le regole alla computazione d'esempio (Figura 4.3); i vettori hanno tre componenti, una per processo.

- e_1^1 , e_2^1 ed e_3^1 sono i primi eventi dei rispettivi processi, con vector clock $(1, 0, 0)$, $(0, 1, 0)$ e $(0, 0, 1)$.
- e_1^2 riceve la richiesta di p_2 , con timestamp $(0, 1, 0)$: il massimo con $(1, 0, 0)$ è $(1, 1, 0)$, e incrementando la componente di p_1 si ottiene $(2, 1, 0)$.
- e_3^2 riceve la richiesta di p_1 , con timestamp $(1, 0, 0)$: dal massimo con $(0, 0, 1)$, cioè $(1, 0, 1)$, si ottiene $(1, 0, 2)$. Seguono due invii, con vector clock $(1, 0, 3)$ e $(1, 0, 4)$.
- e_1^3 riceve la risposta di p_3 , con timestamp $(1, 0, 3)$: dal massimo con $(2, 1, 0)$, cioè $(2, 1, 3)$, si ottiene $(3, 1, 3)$. Gli eventi successivi di p_1 non

ricevono nulla e incrementano solo la prima componente: $(4, 1, 3)$, $(5, 1, 3)$, $(6, 1, 3)$.

- e_2^2 riceve la richiesta di p_3 , con timestamp $(1, 0, 4)$, e vale $(1, 2, 4)$; e_2^3 riceve la risposta di p_1 , con timestamp $(4, 1, 3)$: il massimo con $(1, 2, 4)$ è $(4, 2, 4)$, quindi $(4, 3, 4)$.
- e_3^5 è interno, $(1, 0, 5)$; e_3^6 riceve la richiesta di p_1 , con timestamp $(5, 1, 3)$, e vale $(5, 1, 6)$.

Per esempio $VC(e_2^3) = (4, 3, 4)$ dice che e_2^3 è il terzo evento di p_2 e che lo precedono causalmente quattro eventi di p_1 (e_1^1 – e_1^4) e quattro di p_3 (e_3^1 – e_3^4).

4.5 Confrontare i vector clock

Due vettori V e V' di n naturali si confrontano componente per componente:

$$V \leq V' \iff V[k] \leq V'[k] \text{ per ogni } k, \quad V < V' \iff V \leq V' \text{ e } V \neq V'.$$

Due vettori possono anche essere *incomparabili*, come $(2, 1, 0)$ e $(1, 0, 4)$: il primo è più grande nella prima componente, il secondo nella terza. È proprio ciò che serve per rappresentare la concorrenza. Seguono le proprietà principali dei vector clock, tutte conseguenze dirette delle regole di aggiornamento.

Strong clock condition. Per ogni coppia di eventi

$$e \rightarrow e' \iff VC(e) < VC(e').$$

Per esempio $VC(e_2^1) = (0, 1, 0) < (5, 1, 6) = VC(e_3^6)$, quindi $e_2^1 \rightarrow e_3^6$; invece $VC(e_2^2) = (1, 2, 4)$ e $VC(e_3^6) = (5, 1, 6)$ sono incomparabili, quindi i due eventi sono concorrenti. Per questo test non serve sapere su quali processi sono avvenuti gli eventi.

Test con una sola componente. Se si sa su quali processi sono avvenuti gli eventi basta un confronto: per un evento e_i di p_i e un evento e_j di p_j , con $i \neq j$,

$$e_i \rightarrow e_j \iff VC(e_i)[i] \leq VC(e_j)[i].$$

L'idea: $VC(e_j)[i]$ dice quanti eventi di p_i precedono e_j , e $VC(e_i)[i]$ è la posizione di e_i tra gli eventi di p_i ; e_i precede e_j se e solo se è tra quelli. Per esempio $VC(e_1^4)[1] = 4 \leq 4 = VC(e_2^3)[1]$, quindi $e_1^4 \rightarrow e_2^3$. L'uguaglianza indica che e_i è l'ultimo evento di p_i che precede e_j , e quindi è un invio: qui e_1^4 invia proprio la risposta che p_2 riceve in e_2^3 .

Concorrenza. Ne segue un test immediato per la concorrenza:

$$e_i \parallel e_j \iff VC(e_i)[i] > VC(e_j)[i] \text{ e } VC(e_j)[j] > VC(e_i)[j],$$

cioè nessuno dei due eventi è tra quelli che l'altro conosce. Per e_2^2 ed e_3^6 : $VC(e_2^2)[2] = 2 > 1 = VC(e_3^6)[2]$ e $VC(e_3^6)[3] = 6 > 4 = VC(e_2^2)[3]$.

Tagli consistenti. Con i vector clock si controlla anche la consistenza di un taglio, guardando solo la sua frontiera. Un taglio $(c_1, \dots, c_n)$ è consistente se e solo se

$$VC(e_i^{c_i})[i] \geq VC(e_j^{c_j})[i] \quad \text{per ogni } i, j,$$

cioè nessun evento della frontiera conosce di p_i più eventi di quanti p_i ne abbia nel taglio. Se invece $VC(e_j^{c_j})[i] > c_i$, l'evento $e_j^{c_j}$ dipende da un evento di p_i rimasto fuori dal taglio: $e_i^{c_i}$ ed $e_j^{c_j}$ sono **inconsistenti a coppie** (*pairwise inconsistent*), cioè non possono stare insieme sulla frontiera di un taglio consistente. La frontiera del taglio $C' = (3, 2, 6)$ della Figura 2.1 è formata da e_1^3 , e_2^2 ed e_3^6 , con vector clock $(3, 1, 3)$, $(1, 2, 4)$ e $(5, 1, 6)$: e_3^6 conosce 5 eventi di p_1 , ma il taglio ne contiene solo 3, quindi C' è inconsistente. Per $C = (5, 2, 4)$, con vector clock di frontiera $(5, 1, 3)$, $(1, 2, 4)$ e $(1, 0, 4)$, la condizione vale per ogni coppia. Due eventi legati causalmente possono comunque stare sulla stessa frontiera, per esempio un invio e la ricezione corrispondente: conta solo che nessun evento della frontiera dipenda da eventi esclusi.

Weak gap detection. Infine i vector clock offrono una forma debole della gap detection che mancava ai clock logici. Se $VC(e_i)[k] < VC(e_j)[k]$ per qualche $k \neq j$, allora esiste un evento e_k di p_k tale che

$$e_k \not\rightarrow e_i \quad \text{e} \quad e_k \rightarrow e_j.$$

In parole: se e_j conosce più eventi di p_k di quanti ne conosca e_i , c'è un evento di p_k nel passato di e_j ma non in quello di e_i . La proprietà è «debole» perché in generale non si può concludere che i tre eventi formino una catena $e_i \rightarrow e_k \rightarrow e_j$. Lo si può concludere però quando $k = i$: se $VC(e_i)[i] < VC(e_j)[i]$, c'è un evento di p_i successivo a e_i che precede e_j , cioè un evento che cade nel «buco» tra i due. È questo il caso che servirà al monitor per accorgersi che gli manca una notifica.

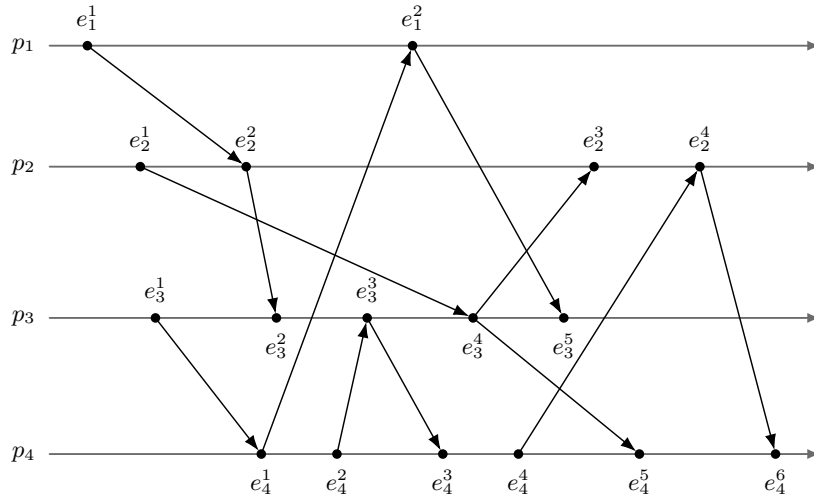


Figura 4.4. Una computazione di quattro processi, da etichettare con i clock logici e con i vector clock.

Tabella 4.1. Clock logici e vector clock della computazione della Figura 4.4. La colonna centrale indica l'invio di cui l'evento riceve il messaggio.

Evento	Riceve da	LC	VC
e_1^1	—	1	(1, 0, 0, 0)
e_1^2	e_4^1	3	(2, 0, 1, 1)
e_2^1	—	1	(0, 1, 0, 0)
e_2^2	e_1^1	2	(1, 2, 0, 0)
e_2^3	e_3^4	6	(1, 3, 4, 2)
e_2^4	e_4^4	7	(1, 4, 4, 4)
e_3^1	—	1	(0, 0, 1, 0)
e_3^2	e_2^2	3	(1, 2, 2, 0)
e_3^3	e_4^2	4	(1, 2, 3, 2)
e_3^4	e_2^1	5	(1, 2, 4, 2)
e_3^5	e_1^2	6	(2, 2, 5, 2)
e_4^1	e_3^1	2	(0, 0, 1, 1)
e_4^2	—	3	(0, 0, 1, 2)
e_4^3	e_3^3	5	(1, 2, 3, 3)
e_4^4	—	6	(1, 2, 3, 4)
e_4^5	e_3^4	7	(1, 2, 4, 5)
e_4^6	e_2^4	8	(1, 4, 4, 6)

Esempio (clock logici e vector clock a confronto). Etichettiamo con entrambi i clock la computazione della Figura 4.4, con quattro processi. Come nell'esercizio da cui è presa, un evento che riceve un messaggio e ne invia subito altri conta come un solo evento: si applica la regola della ricezione, e il valore ottenuto è anche il timestamp dei messaggi inviati. Per esempio e_3^4 riceve il messaggio di e_2^1 e invia a p_2 e a p_4 , ed e_4^1 riceve da e_3^1 e invia a p_1 . La Tabella 4.1 riporta i risultati. Alcune osservazioni:

- e_4^1 riceve da e_3^1 , che ha clock logico 1, quindi vale $\max\{0, 1\} + 1 = 2$: una ricezione porta sempre il clock oltre il timestamp del messaggio, e di conseguenza e_4^2 vale 3.
- e_3^2 riceve da e_2^2 , con timestamp (1, 2, 0, 0): il massimo con (0, 0, 1, 0) è (1, 2, 1, 0), e incrementando la componente di p_3 si ottiene (1, 2, 2, 0). È facile dimenticare l'incremento finale.
- e_1^2 ed e_3^3 hanno clock logici crescenti, 3 e 4, ma i loro vector clock (2, 0, 1, 1) e (1, 2, 3, 2) sono incomparabili: gli eventi sono concorrenti. Lo stesso vale per e_3^4 ed e_4^4 , con clock logici 5 e 6. Il clock logico non distingue gli eventi concorrenti da quelli legati, il vector clock sì.
- Con il test a una componente si vede subito che $e_1^1 \rightarrow e_4^6$: $VC(e_1^1)[1] = 1 \leq 1 = VC(e_4^6)[1]$.

4.6 Consegna causale con i vector clock

Torniamo al monitor. I processi gli notificano i propri eventi, e ogni notifica m porta come timestamp $TS(m)$ il vector clock dell'evento notificato. Supponiamo che i processi incrementino la propria componente solo negli eventi notificati al monitor (oppure, che è lo stesso, che notifichino tutti i loro eventi): così $TS(m)[k]$ conta le notifiche di p_k che precedono causalmente m . Il monitor tiene da parte i messaggi ricevuti ma non ancora consegnati, e un array di contatori $D[1 \dots n]$, inizialmente a zero: $D[k]$ è il numero di notifiche di p_k già consegnate, cioè il valore $TS[k]$ dell'ultima notifica consegnata da p_k .

Una notifica m di p_j si può consegnare quando p_0 è sicuro che non esiste nessun'altra notifica, già arrivata o ancora in viaggio, il cui invio preceda causalmente quello di m . Le condizioni da verificare sono due.

1. *Non manca nessuna notifica precedente di p_j .* m deve essere proprio la prossima notifica di p_j , cioè p_0 deve averne già consegnate $TS(m)[j] - 1$.
2. *Non manca nessuna notifica di un altro processo che preceda m .* Quando ha inviato m , p_j conosceva $TS(m)[k]$ notifiche di p_k , e p_0 deve averle già consegnate tutte. Se $D[k] < TS(m)[k]$, per la weak gap detection esiste una notifica di p_k che precede causalmente m e non è ancora stata consegnata.

Definizione (regola DR3, consegna causale). Consegna la notifica m di p_j non appena

$$D[j] = TS(m)[j] - 1 \quad \text{e} \quad D[k] \geq TS(m)[k] \quad \text{per ogni } k \neq j,$$

e dopo la consegna poni $D[j] := TS(m)[j]$.

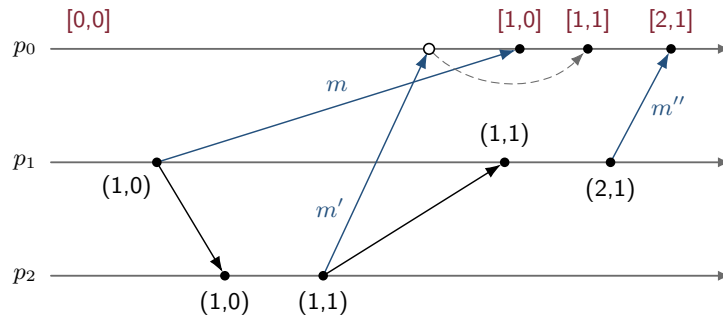


Figura 4.5. La regola DR3 al lavoro. Su p_1 e p_2 i vector clock, su p_0 (in rosso) i valori di D ; in blu le notifiche. m' arriva prima di m (cerchio vuoto) e viene consegnata solo dopo di essa.

Esempio (la regola DR3 al lavoro). Nella Figura 4.5 ci sono due processi, p_1 e p_2 , e il monitor p_0 . I vector clock hanno due componenti, una per processo, e si incrementano solo negli eventi notificati.

- p_1 esegue un evento notificato, con vector clock $(1, 0)$, in cui invia un messaggio a p_2 ; la sua notifica è m .
- p_2 riceve il messaggio. L'evento non è notificato, quindi il vector clock diventa $\max\{(0, 0), (1, 0)\} = (1, 0)$ senza incremento. Poi p_2 esegue un evento notificato, con vector clock $(1, 1)$, in cui invia un messaggio a p_1 ; la sua notifica è m' .
- m' arriva a p_0 per prima. Con $D = [0, 0]$ e $TS(m') = (1, 1)$, la prima condizione vale ($D[2] = 0 = 1 - 1$) ma la seconda no ($D[1] = 0 < 1$): m' dipende da un evento di p_1 di cui p_0 non ha ancora visto la notifica, e resta in attesa.
- Arriva m , con $TS(m) = (1, 0)$: $D[1] = 0 = 1 - 1$ e $D[2] = 0 \geq 0$, quindi m viene consegnata e D diventa $[1, 0]$. Ora anche m' soddisfa entrambe le condizioni ($D[2] = 0 = 1 - 1$ e $D[1] = 1 \geq 1$): viene consegnata e D diventa $[1, 1]$.
- p_1 riceve il messaggio di p_2 (vector clock $(1, 1)$) e poi esegue un evento notificato, $(2, 1)$, con notifica m'' . Quando m'' arriva, $D[1] = 1 = 2 - 1$ e $D[2] = 1 \geq 1$: viene consegnata subito, e D diventa $[2, 1]$.

Con la regola DR3 si ha una soluzione completa di GPE con monitoraggio passivo in un sistema asincrono. I processi notificano a p_0 gli eventi rilevanti; p_0 consegna le notifiche con una regola di consegna causale e ottiene un'osservazione che corrisponde a un run consistente; Φ si può valutare in qualunque stato globale del run, perché sono tutti consistenti.

La stessa regola funziona in qualunque processo, non solo nel monitor. Se i processi comunicano solo con *broadcast*, cioè inviando ogni messaggio a tutti, DR3 garantisce la consegna causale in ogni destinatario: è il *causal broadcast*, un'astrazione molto usata per costruire applicazioni distribuite. Con messaggi punto a punto servono invece timestamp più ricchi, matrici $n \times n$ formate da n vector clock.

Canali nascosti. La consegna causale permette di ragionare sul sistema globale usando solo informazioni locali, ma le conclusioni valgono solo in un **sistema chiuso**, in cui tutta la comunicazione passa per i canali del sistema. Se esistono **canali nascosti** (*hidden channels*), esterni al sistema, la precedenza causale non li vede. Un esempio: p_1 e p_2 sorvegliano lo stesso tubo del vapore in due punti diversi, e p_3 controlla il riscaldamento. p_1 rileva la rottura del tubo e la notifica a p_3 ; poco dopo p_2 , più a valle, rileva un calo di pressione e notifica anche quello. Per il sistema le due notifiche sono concorrenti, quindi p_3 può consegnare per prima quella del calo di pressione, reagire aumentando il calore, e solo dopo ricevere la notizia della rottura: concluderebbe che il tubo si è rotto per il calore, mentre è vero il contrario. Il tubo stesso fa da canale di comunicazione esterno: la rottura causa il calo di pressione, ma la relazione $\rightarrow$ non lo sa. Nei sistemi che non sono chiusi bisogna ragionare su osservazioni ordinate dal tempo reale, che rispetta anche questi legami.

Capitolo 5

Snapshot distribuiti

Si torna alla strategia attiva del capitolo 2: il monitor «fotografa» gli stati dei processi e li mette insieme. Lì il risultato poteva essere inconsistente; qui si costruisce un protocollo di **snapshot** (istantanea) che produce solo stati globali consistenti, senza fermare la computazione. È il protocollo di Chandy e Lamport, che si ricava passo passo partendo da un sistema sincrono e togliendo un'ipotesi alla volta. Si assume che i canali siano FIFO.

5.1 Stati dei canali

Uno snapshot registra gli stati locali dei processi e i messaggi in viaggio. Le difficoltà sono due: gli stati dei processi non si possono registrare tutti nello stesso istante, e i messaggi in viaggio non li vede nessuno.

Definizione (stato di un canale). Lo **stato del canale** da p_i a p_j , indicato con $\chi_{i,j}$, è l'insieme dei messaggi che p_i ha inviato a p_j e che p_j non ha ancora ricevuto.

Come si è visto nel capitolo 2, lo stato di un canale si può ricavare dagli stati locali, come differenza tra i messaggi inviati da p_i a p_j (registrati in σ_i) e quelli ricevuti da p_j (registrati in σ_j). Registrarlo esplicitamente però è comodo: gli stati locali restano più compatti e i predicati più semplici. Per esempio, nel grafo delle attese p_i è bloccato su p_j se σ_i registra una richiesta a p_j ancora aperta e $\chi_{j,i}$ non contiene risposte.

Indichiamo con IN_i l'insieme dei processi che hanno un canale verso p_i (i canali **entranti** di p_i) e con OUT_i quello dei processi verso cui p_i ha un canale (i suoi canali **uscenti**). In uno snapshot ogni processo p_i registra il proprio stato locale σ_i e gli stati $\chi_{j,i}$ di tutti i suoi canali entranti.

5.2 Primo protocollo: un orologio reale

Si parte, come al solito, da ipotesi forti: tutti i processi leggono un orologio reale globale RC , i ritardi dei messaggi sono limitati da un valore noto, e lo sono anche le velocità relative dei processi. L'idea è che tutti registrino lo stato nello stesso

istante reale t_{ss} , scelto da p_0 abbastanza avanti nel tempo da essere sicuro che il suo annuncio arrivi a tutti prima di t_{ss} . Ogni messaggio porta come timestamp l'istante in cui è stato inviato.

Definizione (Protocollo 1, con orologio reale).

1. p_0 invia a tutti i processi il messaggio «take snapshot at t_{ss} ».
2. Quando l'orologio segna t_{ss} , ogni processo p_i registra il proprio stato locale σ_i , invia un messaggio vuoto su tutti i canali uscenti e comincia a registrare i messaggi che riceve su ogni canale entrante. Registrazione e invio dei messaggi vuoti avvengono prima di qualunque altro evento della computazione.
3. La prima volta che p_i riceve da p_j un messaggio con timestamp maggiore o uguale a t_{ss} , smette di registrare quel canale: $\chi_{j,i}$ è l'insieme dei messaggi registrati.

Lo stato del canale $\chi_{j,i}$ contiene quindi i messaggi inviati da p_j prima di t_{ss} e ricevuti da p_i dopo t_{ss} : proprio quelli in viaggio nell'istante dello snapshot. I messaggi vuoti servono alla liveness: garantiscono che su ogni canale entrante arrivi prima o poi un messaggio con timestamp almeno t_{ss} , e quindi che la registrazione di ogni canale finisca.

Lo stato costruito è consistente, ed è anzi uno stato che si è verificato davvero: è una fotografia scattata da tutti nello stesso istante. Conviene però vedere quale proprietà dell'orologio garantisce la consistenza. Un evento e sta nel taglio C_{ss} dello snapshot se e solo se $RC(e) < t_{ss}$. Se $e \in C_{ss}$ ed $e' \rightarrow e$, per la clock condition $RC(e') < RC(e) < t_{ss}$, quindi anche $e' \in C_{ss}$: il taglio è consistente. L'unica proprietà dell'orologio reale che serve è, ancora una volta, la clock condition.

Esempio (snapshot in un sistema sincrono). Un esercizio chiede come registrare uno stato globale, con un algoritmo più semplice di quello di Chandy–Lamport, in un sistema in cui gli orologi fisici di tutti i nodi sono perfettamente sincronizzati e la rete è affidabile. La risposta è il Protocollo 1. Se Δ è il tempo massimo perché un messaggio raggiunga tutti i nodi, p_0 annuncia a tutti «take snapshot at $t + \Delta$ », dove t è l'istante attuale, e ogni processo registra il proprio stato quando il suo orologio segna $t + \Delta$. Se serve anche lo stato dei canali, lo si registra come al passo 3 del protocollo.

5.3 Dall'orologio reale al protocollo di Chandy–Lamport

Poiché serve solo la clock condition, viene naturale sostituire l'orologio reale con i clock logici, che la soddisfano anche in un sistema asincrono. Ci sono però due problemi.

- «Quando il clock segna t_{ss} » non ha più senso: un clock logico può saltare un valore (nella Figura 3.1 quello di p_3 passa da 5 a 7), e cambia solo quando avvengono eventi. Il problema si risolve con alcune regole tecniche sull'ordine di esecuzione, che qui si omettono.

- p_0 non può scegliere un valore logico che tutti raggiungano solo dopo aver ricevuto il suo annuncio. Si assume allora un valore ω così grande che nessun clock lo raggiunge con le normali regole di aggiornamento. Anche questa ipotesi, però, richiede di limitare velocità e ritardi, e andrà tolta.

Nasce così il Protocollo 2: p_0 invia «take snapshot at ω » e porta il proprio clock a ω ; ogni processo registra il proprio stato quando il suo clock raggiunge ω , e i messaggi vuoti, con timestamp ω , portano a ω anche i clock dei vicini. Poiché la rete è fortemente connessa, prima o poi tutti i clock raggiungono ω e il protocollo termina.

Ora si nota che tra l'arrivo di «take snapshot at ω » e l'arrivo del primo messaggio vuoto, che porta il clock a ω , un processo non fa nulla di utile per il protocollo. Si può quindi eliminare l'annuncio e far registrare lo stato all'arrivo del primo messaggio vuoto. Per non confonderlo con altri messaggi vuoti, lo si trasforma in un messaggio speciale, «take snapshot», spesso chiamato **marker**. Infine non servono più nemmeno i timestamp: con canali FIFO il marker è il primo messaggio che il mittente invia dopo la propria registrazione, e quindi separa su ogni canale i messaggi inviati «prima» dello snapshot da quelli inviati «dopo». Senza timestamp spariscono anche i clock logici, e resta il protocollo di Chandy–Lamport.

5.4 Il protocollo di Chandy–Lamport

Definizione (protocollo di Chandy–Lamport).

1. p_0 avvia il protocollo inviando a sé stesso un messaggio «take snapshot».
2. Sia p_f il processo da cui p_i riceve «take snapshot» per la prima volta. Appena lo riceve, p_i registra il proprio stato locale σ_i e inoltra «take snapshot» su tutti i suoi canali uscenti, senza eseguire eventi della computazione tra le due cose. Pone $\chi_{f,i} = \emptyset$ e comincia a registrare i messaggi che riceve su ciascuno degli altri canali entranti.
3. Sia p_s un processo da cui p_i riceve «take snapshot» dopo la prima volta. p_i smette di registrare il canale da p_s , e $\chi_{s,i}$ è l'insieme dei messaggi registrati su quel canale.

Il marker viene inoltrato solo alla prima ricezione e la rete è fortemente connessa, quindi attraversa ogni canale esattamente una volta. Quando p_i ha ricevuto il marker da tutti i canali entranti, il suo contributo allo stato globale è completo e la sua parte del protocollo è finita; resta da inviare a p_0 lo stato locale e gli stati dei canali, un dettaglio che qui si trascura.

Perché i canali vengono registrati così? Sul canale da p_f il marker separa i messaggi inviati da p_f prima della sua registrazione da quelli inviati dopo. Quando il marker arriva, i messaggi «precedenti» sono già arrivati tutti e fanno parte di σ_i , quindi nel canale non resta nulla: $\chi_{f,i} = \emptyset$. Su un altro canale entrante, da p_s , i messaggi che arrivano dopo la registrazione di p_i ma prima del marker di p_s sono stati inviati da p_s prima di registrarsi e ricevuti da p_i dopo essersi registrato: sono esattamente quelli in viaggio «nell'istante» dello snapshot.

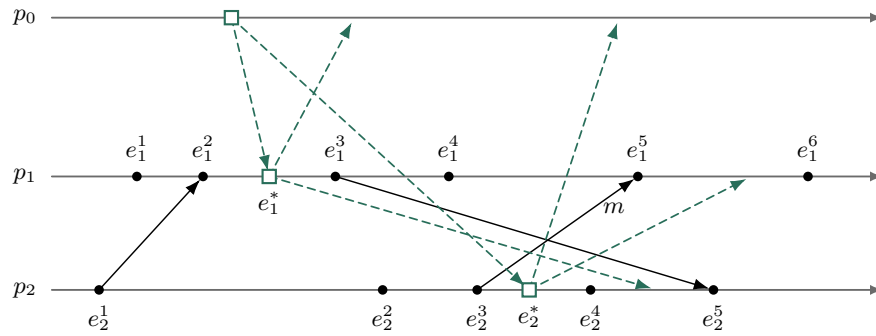


Figura 5.1. Il protocollo di Chandy–Lamport sulla computazione della Figura 2.3. Trattaggiati i marker; i quadratini segnano le registrazioni (e_1^* ed e_2^*).

Esempio (il protocollo passo passo). Nella Figura 5.1 il protocollo viene eseguito sulla computazione di due processi della Figura 2.3.

- p_0 avvia il protocollo e invia il marker a p_1 e a p_2 .
- p_1 riceve il marker di p_0 dopo e_1^2 : registra σ_1^2 , inoltra il marker a p_0 e a p_2 , pone $\chi_{0,1} = \emptyset$ e comincia a registrare il canale da p_2 .
- Il marker di p_0 arriva a p_2 solo dopo e_2^3 : p_2 registra σ_2^3 , inoltra il marker, pone $\chi_{0,2} = \emptyset$ e comincia a registrare il canale da p_1 .
- Il messaggio m , inviato da p_2 in e_2^3 , prima della sua registrazione, arriva a p_1 in e_1^5 , dopo la registrazione di p_1 ma prima del marker di p_2 : viene registrato. Quando poi arriva il marker di p_2 , $\chi_{2,1} = \{m\}$.
- Il marker di p_1 arriva a p_2 prima del messaggio inviato in e_1^3 , che è partito dopo il marker e non può sorpassarlo: $\chi_{1,2} = \emptyset$.

Lo stato costruito è Σ^{23} , con $\chi_{1,2}$ vuoto e $\chi_{2,1} = \{m\}$. È consistente: l'unico messaggio ricevuto dentro il taglio, quello inviato in e_2^1 , è stato inviato dentro il taglio, e m , inviato dentro e ricevuto fuori, compare correttamente tra i messaggi in viaggio.

5.5 Perché funziona, e perché servono i canali FIFO

Lo stato costruito dal protocollo è sempre consistente, e l'idea è semplice. Nel capitolo 2 si è visto che un taglio è inconsistente solo se c'è un messaggio ricevuto dentro il taglio ma inviato fuori. Supponiamo che p_j invii un messaggio a p_k *dopo* essersi registrato, cioè fuori dal taglio. Al momento della registrazione p_j ha inviato il marker su tutti i canali uscenti, compreso quello verso p_k ; sul canale quindi il marker precede il messaggio, e con canali FIFO arriva prima. Quando il messaggio arriva, p_k si è già registrato, e anche la ricezione è fuori dal taglio. Nessun messaggio arriva «dal futuro», quindi il taglio è consistente.

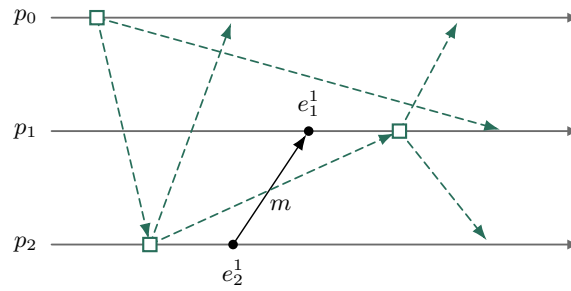


Figura 5.2. Chandy-Lamport senza canali FIFO: m , inviato da p_2 dopo la registrazione, sorpassa il marker, e p_1 lo riceve prima di registrarsi.

Esempio (senza FIFO il protocollo non funziona). Nella Figura 5.2 il marker di p_0 raggiunge subito p_2 , che si registra prima di qualunque evento e inoltra il marker a p_0 e a p_1 . Subito dopo p_2 invia a p_1 il messaggio m (e_2^1). Il canale da p_2 a p_1 non è FIFO, e m sorpassa il marker: p_1 lo riceve (e_1^1) quando non si è ancora registrato, e si registra solo all'arrivo del marker di p_2 , il primo che riceve. Lo stato costruito comprende la ricezione di m , perché e_1^1 è nel taglio, ma non il suo invio, perché e_2^1 ne è fuori: è inconsistente.

Il protocollo si può estendere in molti modi, per esempio per farlo funzionare senza canali FIFO o per ridurre il numero di messaggi.

5.6 Proprietà dello snapshot

Indichiamo con Σ^a lo stato globale in cui il protocollo viene avviato, con Σ^f quello in cui termina e con Σ^s lo stato costruito dallo snapshot. Σ^s è consistente, ma non è detto che il sistema ci sia mai passato davvero.

Esempio (uno snapshot mai accaduto). Nell'esecuzione della Figura 5.1 il run seguito davvero dai processi è

$$e_2^1 e_1^1 e_1^2 e_1^3 e_2^2 e_1^4 e_2^3 e_2^4 e_1^5 e_2^5 e_1^6,$$

cioè, in termini di stati globali,

$$\Sigma^{00} \Sigma^{01} \Sigma^{11} \Sigma^{21} \Sigma^{31} \Sigma^{32} \Sigma^{42} \Sigma^{43} \Sigma^{44} \Sigma^{54} \Sigma^{55} \Sigma^{65}.$$

Il protocollo parte in Σ^{21} , finisce in Σ^{55} e costruisce Σ^{23} , che nel run non compare: quando p_1 si è registrato, p_2 aveva eseguito un solo evento, e quando si è registrato p_2 , p_1 era già arrivato a e_1^4 . Il caso più semplice è quello di due processi con un solo evento interno ciascuno, senza messaggi, in cui e_2^1 avviene prima di e_1^1 : se il marker arriva a p_2 presto e a p_1 tardi, lo snapshot registra Σ^{10} , mentre il sistema è passato per Σ^{00} , Σ^{01} e Σ^{11} .

Lo stato costruito non è però uno stato consistente qualunque: si colloca sempre tra l'inizio e la fine del protocollo,

$$\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f,$$

cioè esiste un run consistente che passa per lo stato di avvio, poi per lo stato costruito e infine per quello di arrivo. Nell'esempio $\Sigma^{21} \rightsquigarrow \Sigma^{23} \rightsquigarrow \Sigma^{55}$, come si vede nel reticolo (Figura 5.3).

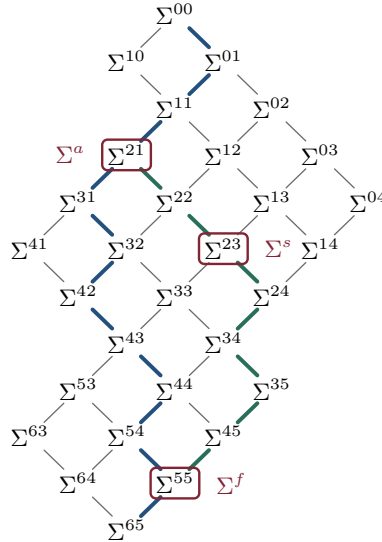


Figura 5.3. Lo snapshot nel reticolo della Figura 2.3. Il run effettivo (blu) passa per l'avvio $\Sigma^a = \Sigma^{21}$ e la fine $\Sigma^f = \Sigma^{55}$, ma non per lo stato costruito $\Sigma^s = \Sigma^{23}$; un altro run consistente (verde) passa per tutti e tre.

L'idea si vede bene guardando i tagli. Il taglio dello stato di avvio è contenuto in quello dello snapshot, perché ogni processo si registra dopo l'avvio del protocollo; e il taglio dello snapshot è contenuto in quello dello stato di arrivo, perché ogni processo si registra prima della fine. I tre tagli sono consistenti (i due estremi perché si sono verificati davvero), e per una proprietà vista nella Sezione 2.4 lo stato di un taglio consistente è raggiungibile da quello di ogni taglio consistente che vi è contenuto. Un altro modo di vederla: un evento successivo alla registrazione del proprio processo non può precedere causalmente un evento anteriore alla registrazione del suo, per lo stesso argomento sui canali FIFO. Nel run reale si possono quindi spostare tutti gli eventi anteriori alle registrazioni prima di quelli successivi senza violare la precedenza causale, e il run che si ottiene passa proprio per Σ^s .

5.7 A cosa serve uno snapshot

Se lo stato costruito può non essere mai esistito, e in ogni caso descrive il passato, a che cosa serve? Serve a rilevare le proprietà **stabili**: quelle che, una volta diventate vere, restano vere per sempre. Sono stabili, per esempio, il deadlock, la terminazione, la perdita di tutti i token e il fatto che un oggetto non sia più raggiungibile da nessuno. Per un predicato stabile Φ , da $\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f$ seguono due conclusioni:

- se Φ è vero in Σ^s , è vero anche in Σ^f e in tutti gli stati successivi: la proprietà vale ora;
- se Φ è falso in Σ^s , era falso anche in Σ^a : quando il protocollo è partito, la proprietà non valeva ancora.

Uno snapshot permette quindi di rilevare correttamente deadlock e terminazione e di salvare un punto di ripristino consistente da cui ripartire dopo un guasto. Permette anche di calcolare grandezze che si conservano, come il denaro complessivo in un sistema bancario: in uno stato consistente, i saldi dei conti più i trasferimenti in viaggio danno il totale giusto.

Esempio (uno snapshot con i canali vuoti). Un esercizio chiede di modificare il protocollo perché registri uno snapshot «fortemente consistente», in cui tutti i canali sono vuoti. L'idea è fermare gli invii, svuotare i canali e solo allora registrare.

1. p_0 invia a tutti il messaggio STOP.
2. Ricevuto STOP, p_i smette di inviare messaggi della computazione (può ancora riceverne ed eseguire eventi interni) e invia un messaggio ACK su ogni canale uscente.
3. Quando p_i ha ricevuto ACK da tutti i canali entranti, ha già ricevuto tutti i messaggi che gli erano stati inviati, perché con canali FIFO l'ACK è l'ultimo messaggio su ciascun canale: i suoi canali entranti sono vuoti. p_i registra σ_i e lo invia a p_0 .
4. Quando ha ricevuto tutti gli stati, p_0 invia a tutti RESUME, e i processi riprendono a inviare messaggi.

La ripresa deve aspettare che tutti si siano registrati: se p_i ricominciasse a inviare subito dopo la propria registrazione, un suo messaggio potrebbe arrivare a un processo non ancora registrato, e lo stato costruito conterrebbe una ricezione senza il relativo invio. Il prezzo è che la computazione resta ferma per tutta la durata dello snapshot. Chandy-Lamport, invece, ottiene uno stato consistente senza fermare nulla, e in cambio registra anche i messaggi in viaggio.